

DIFFERENTIALLY PRIVATE ONE PERMUTATION HASHING

Anonymous authors

Paper under double-blind review

ABSTRACT

Minwise hashing (MinHash) is a standard hashing algorithm for large-scale search and learning with the binary Jaccard similarity. One permutation hashing (OPH) is an effective and efficient alternative of MinHash which splits the data into K bins and generates hash values within each bin. In this paper, to protect the privacy of the output sketches, we combine differential privacy (DP) with OPH, and propose DP-OPH framework with three variants: DP-OPH-fix, DP-OPH-re and DP-OPH-rand, depending on the densification strategy to deal with empty bins in OPH. Detailed algorithm design and privacy and utility analysis are provided. The proposed DP-OPH methods significantly improves the DP minwise hashing (DP-MH) alternative in the literature. Experiments on similarity search confirm the effectiveness of our proposed algorithms. We also provide an extension to real-value data, named DP-BCWS, in the appendix.

1 INTRODUCTION

Let $\mathbf{u}, \mathbf{v} \in \{0, 1\}^D$ be two D -dimensional binary vectors. In this paper, we focus on the hashing algorithms for the Jaccard similarity (a.k.a. the “resemblance”) defined as $J(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^D \mathbb{1}\{\mathbf{u}_i = \mathbf{v}_i = 1\}}{\sum_{i=1}^D \mathbb{1}\{\mathbf{u}_i + \mathbf{v}_i \geq 1\}}$. This is a widely used similarity measure in machine learning applications. $\mathbf{u}$ and $\mathbf{v}$ can also be viewed as two sets of items represented by the locations of non-zero entries. In industrial applications with massive data size, directly calculating the pairwise Jaccard similarity among the data points becomes too expensive. To accelerate large-scale search and learning, the celebrated “minwise hashing” (MinHash) algorithm (Broder, 1997; Broder et al., 1997) has been a standard hashing technique for approximating the Jaccard similarity in massive binary datasets. It has seen numerous applications such as near neighbor search, duplicate detection, malware detection, clustering, large-scale learning, social networks, and computer vision (Indyk & Motwani, 1998; Charikar, 2002; Fetterly et al., 2003; Das et al., 2007; Buehrer & Chellapilla, 2008; Bendersky & Croft, 2009; Chierichetti et al., 2009; Pandey et al., 2009; Lee et al., 2010; Deng et al., 2012; Chum & Matas, 2012; Tamersoy et al., 2014; Shrivastava & Li, 2014; Zhu et al., 2017; Nargesian et al., 2018; Wang et al., 2019; Lemiesz, 2021; Feng & Deng, 2021; Li & Li, 2022). The output of MinHash is an integer. For large-scale applications, to store and use the hash values (or called sketches) more conveniently and efficiently, Li & König (2010) proposed b -bit MinHash that only stores the last b bits of the hashed integers, which is memory-efficient and convenient for similarity search and machine learning. Thus, it has been a popular coding strategy for the MinHash values and its alternatives (Li et al., 2011; 2015; Shah & Meinshausen, 2017; Yu & Weber, 2022).

1.1 ONE PERMUTATION HASHING (OPH) FOR JACCARD SIMILARITY APPROXIMATION

To use MinHash in practice, we need to generate K hash values to achieve good utility. This requires applying K random permutations (or hash functions as approximations) per data point, yielding an $O(Kf)$ complexity where f is the number of non-zero elements. One permutation hashing (OPH) (Li et al., 2012) provides a strategy to significantly reduce the complexity to $O(f)$. The idea of OPH is: to generate K hashes, we split the data vector into K non-overlapping bins, and conduct MinHash within each bin. Yet, empty bins may arise which breaks the alignment of the hashes. To deal with empty bins, densification schemes (Shrivastava, 2017; Li et al., 2019) are proposed that fill the empty bins with some non-empty bin. It is shown that OPH with densification provides unbiased Jaccard estimator, and the estimation variance can often be smaller than that of MinHash. OPH has

been widely used as an improved method over MinHash for the Jaccard similarity (Dahlgaard et al., 2017; Zhao et al., 2020; Jia et al., 2021; Tseng et al., 2021; Jiang et al., 2022).

1.2 HASHING/SKETCHING AND DIFFERENTIAL PRIVACY

MinHash and OPH both belong to the broad family of probabilistic hashing/sketching methods designed for various purposes and tasks. Examples of more sketching methods include the random projection (RP) based methods for cosine estimation (Charikar, 2002; Vempala, 2005), the count-sketch (CS) for frequency estimation (Charikar et al., 2004), and the Flajolet-Martin (FM) sketch (Flajolet & Martin, 1985) and HyperLogLog sketch (Flajolet et al., 2007) for cardinality estimation, etc. Since the data sketches produce “summaries” of the original data, sketching/hashing may also cause data privacy leakage. Therefore, protecting the privacy of the data sketches has become an important topic which has gained growing research interests in recent years.

Differential privacy (DP) (Dwork et al., 2006b) has become a popular privacy definition with rigorous mathematical formulation, which has been widely applied to clustering, regression and classification, principle component analysis, matrix completion, optimization, deep learning (Blum et al., 2005; Chaudhuri & Monteleoni, 2008; Feldman et al., 2009; Gupta et al., 2010; Chaudhuri et al., 2011; Kasiviswanathan et al., 2013; Zhang et al., 2012; Abadi et al., 2016; Agarwal et al., 2018; Ge et al., 2018; Wei et al., 2020; Dong et al., 2022), etc. Prior efforts have also been conducted on combining differential privacy with the aforementioned hashing algorithms, e.g., for RP (Blocki et al., 2012; Kenthapadi et al., 2013; Stausholm, 2021), count-sketch (Zhao et al., 2022), and FM sketch (Smith et al., 2020; Dickens et al., 2022). Some works (e.g., Blocki et al. (2012); Smith et al. (2020); Dickens et al. (2022)) assumed “internal randomness”, i.e., the randomness of the hash functions are kept private, and showed that many hashing methods themselves already possess strong DP property under some data conditions. However, this setting is more restrictive in practice as it requires that the hash keys or projection matrices cannot be accessed by any adversary. In another setup (e.g., Kenthapadi et al. (2013); Stausholm (2021); Zhao et al. (2022)), both the randomness of the hash functions and the algorithm outputs are treated as public information, and perturbation mechanisms are developed to make the algorithms differentially private.

Contributions. While prior works have proposed DP algorithms for some sketching methods mentioned earlier, the differential privacy of OPH and MinHash for the Jaccard similarity has not been well studied. In this paper, we mainly focus on the differential privacy of one permutation hashing (OPH), the state-of-the-art framework for hashing the Jaccard similarity. We consider the more practical and general setup where the randomness of the algorithm is external/public.

We develop three variants under the DP-OPH framework, DP-OPH-fix, DP-OPH-re, and DP-OPH-rand, corresponding to fixed densification, re-randomized densification, and no densification for OPH, respectively. We provide detailed algorithm design and privacy analysis for each variant, and compare them with a DP MinHash (DP-MH) method. In our retrieval experiments, we show that the proposed DP-OPH method substantially improves DP-MH, and re-randomized densification is superior over fixed densification in terms of differential privacy. DP-OPH-rand performs the best when ϵ is small, while DP-OPH-re is the most performant in when larger ϵ is allowed. We also extend our algorithms to real-value datasets and develop DP-BCWS algorithm in Appendix A.

2 BACKGROUND: MINHASH, b -BIT CODING, AND DIFFERENTIAL PRIVACY

Algorithm 1 Minwise hashing (MinHash)

Input: Binary vector $\mathbf{u} \in \{0, 1\}^D$; number of hash values K

Output: K MinHash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$

- 1: Generate K independent permutations $\pi_1, \dots, \pi_K: [D] \rightarrow [D]$ with seeds $1, \dots, K$ respectively
 - 2: **for** $k = 1$ to K **do**
 - 3: $h_k(\mathbf{u}) \leftarrow \min_{i: u_i \neq 0} \pi_k(i)$
 - 4: **end for**
-

Minwise hashing (MinHash). The MinHash method is summarized in Algorithm 1. We first generate K independent permutations $\pi_1, \dots, \pi_K: [D] \mapsto [D]$. Here, $[D]$ denotes $\{1, \dots, D\}$. For each permutation, the hash value is the first non-zero location in the permuted vector, i.e., $h_k(\mathbf{u}) =$

$\min_{i:v_i \neq 0} \pi_k(i), \forall k = 1, \dots, K$. Analogously, for another data vector $\mathbf{v} \in \{0, 1\}^D$, we also obtain K hash values, $h_k(\mathbf{v})$. The MinHash estimator of $J(\mathbf{u}, \mathbf{v})$ is the average over the hash collisions:

$$\hat{J}_{MH}(\mathbf{u}, \mathbf{v}) = \frac{1}{K} \sum_{k=1}^K \mathbb{1}\{h_k(\mathbf{u}) = h_k(\mathbf{v})\}, \quad (1)$$

where $\mathbb{1}\{\cdot\}$ is the indicator function. By standard probability calculation, we can show that $\mathbb{E}[\hat{J}_{MH}] = J$ and $Var[\hat{J}_{MH}] = \frac{J(1-J)}{K}$. In practice, K does not need to be very large to achieve good utility. For instance, usually $128 \sim 1024$ hash values would be sufficient for search and learning problems (Indyk & Motwani, 1998; Li et al., 2011; Shrivastava & Li, 2014).

b -bit coding of the hash value. Li & König (2010) proposed “ b -bit minwise hashing” as a convenient coding strategy for the integer hash value $h(\mathbf{u})$ generated by MinHash (or by OPH which will be introduced later). Basically, we only keep the last b -bits of each hash value. In our analysis, for convenience, we assume that “taking the last b -bits” can be achieved by some “rehashing” trick to map the integer values onto $\{0, \dots, 2^b - 1\}$ uniformly. There are at least three benefits of this coding strategy: (i) storing only b bits saves the storage cost compared with storing the full 32 or 64 bit integers; (ii) the last few bits are more convenient for the purpose of indexing, e.g., in approximate nearest neighbor search (Indyk & Motwani, 1998); (iii) we can transform the last few bits into a positional representation, allowing us to approximate the Jaccard similarity by inner product, which is required by training large-scale linear models (Li et al., 2011). Given these advantages, in this work, we will adopt this b -bit coding strategy in our private algorithm design.

Differential privacy (DP). We formally define differential privacy (DP) as follows.

Definition 2.1 (Differential privacy (Dwork et al., 2006b)). For a randomized algorithm $\mathcal{M} : \mathcal{U} \mapsto Range(\mathcal{M})$ and $\epsilon, \delta \geq 0$, if for any two neighboring datasets U and U' , the following holds

$$Pr[\mathcal{M}(U) \in Z] \leq e^\epsilon Pr[\mathcal{M}(U') \in Z] + \delta$$

for $\forall Z \subset Range(\mathcal{M})$, then algorithm $\mathcal{M}$ is said to satisfy (ϵ, δ) -DP. If $\delta = 0$, $\mathcal{M}$ is called ϵ -DP.

Intuitively, DP requires that the distributions of the outputs before and after a small change in the data are close, so that an adversary cannot detect the change based on the outputs. Smaller ϵ and δ implies stronger privacy. The parameter δ is usually interpreted as the “failure probability” allowed for the ϵ -DP guarantee to be violated.

Privacy statement and applications. We follow the standard attribute-level DP setup in aforementioned related works on DP hashing/sketching: $\mathbf{u}, \mathbf{u}' \in \{0, 1\}^D$ are called neighboring if they differ in one dimension. Treating the binary vectors as sets, with our proposed DP-OPH algorithms, *an adversary cannot detect from the output sketches whether any item exists in the set or not, which holds independently for all the data vectors in the database*. DP-OPH can naturally be applied as a private variant of OPH in cases where MinHash-type methods are found to be useful. As a concrete example application, the bioinformatics community releases sets of MinHashes for all known genomes on a regular basis (Ondov et al., 2016; Brown & Irber, 2016), which are used for downstream ML tasks like similarity search, classification, clustering, etc. (Berlin et al., 2015) In this type of data, each data point corresponds to (a large set of) genes of a human, which contains the biological information of an individual which is highly sensitive and confidential. Our methods protect the identification of any gene from the released sketches in the DP sense.

3 HASHING FOR JACCARD SIMILARITY WITH DIFFERENTIAL PRIVACY

In this section, we present DP-OPH algorithms based on privatizing the b -bit hash values from OPH and utility analysis, and demonstrate its advantage over a DP-MinHash alternative.

3.1 ONE PERMUTATION HASHING (OPH)

Algorithm 2 outlines the steps of OPH: we first use a permutation π (same for all data vectors) to randomly split the feature dimensions $[D]$ into K bins $\mathcal{B}_1, \dots, \mathcal{B}_K$ with equal length $d = D/K$ (assuming integer division holds). Then, for each bin $\mathcal{B}_k$, we set the smallest permuted index of “1” as the k -th OPH hash value. If $\mathcal{B}_k$ is empty (i.e., it does not contain any “1”), we record an “E” representing empty bin. Li et al. (2012) showed that we can construct statistically unbiased Jaccard

Algorithm 2 One Permutation Hashing (OPH)**Input:** Binary vector $\mathbf{u} \in \{0, 1\}^D$; number of hash values K **Output:** K OPH hash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$

- 1: Let $d = D/K$. Use a permutation $\pi : [D] \mapsto [D]$ with fixed seed to randomly split $[D]$ into K equal-size bins $\mathcal{B}_1, \dots, \mathcal{B}_K$, with $\mathcal{B}_k = \{j \in [D] : (k-1)d + 1 \leq \pi(j) \leq kd\}$
- 2: **for** $k = 1$ to K **do**
- 3: **if** Bin $\mathcal{B}_k$ is non-empty **then**
- 4: $h_k(\mathbf{u}) \leftarrow \min_{j \in \mathcal{B}_k, u_j \neq 0} \pi(j)$
- 5: **else**
- 6: $h_k(\mathbf{u}) \leftarrow E$
- 7: **end if**
- 8: **end for**

Algorithm 3 OPH-fix and OPH-re: OPH with fixed and re-randomized densification**Input:** OPH hash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$ each in $[D] \cup \{E\}$; bins $\mathcal{B}_1, \dots, \mathcal{B}_K$; $d = D/K$ **Output:** K densified OPH hash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$

- 1: Let $NonEmptyBin = \{k \in [K] : h_k(\mathbf{u}) \neq E\}$
- 2: **for** $k = 1$ to K **do**
- 3: **if** $h_k(\mathbf{u}) = E$ **then**
- 4: Uniformly randomly select $k' \in NonEmptyBin$
- 5: $h_k(\mathbf{u}) \leftarrow h_{k'}(\mathbf{u})$ ▷ fixed densification
- 6: **Or**
- 7: $MapToIndex = SortedIndex(\pi(\mathcal{B}_k)) + (k' - 1)d$
- 8: $\pi^{(k)} : \pi(\mathcal{B}_{k'}) \mapsto MapToIndex$ ▷ within-bin partial permutation
- 9: $h_k(\mathbf{u}) \leftarrow \min_{j \in \mathcal{B}_{k'}, u_j \neq 0} \pi^{(k)}(\pi(j))$ ▷ re-randomized densification
- 10: **end if**
- 11: **end for**

estimators by ignoring the empty bins. However, this estimator is unstable when the data is relatively sparse; moreover, since empty bins are different for every distinct data vector, the vanilla OPH hash values do not form a metric space (i.e., the hash values of different data points are not aligned).

Densification for OPH. To tackle the issue caused by empty bins, a series of works has been conducted to densify the OPH. The general idea is to “borrow” the data/hash from non-empty bins, with some careful design. In Algorithm 3, we present two recent representatives of OPH densification methods: **fixed densification** (Shrivastava, 2017) and **re-randomized densification** (Li et al., 2019), noted as OPH-fix and OPH-re, respectively. Given an OPH hash vector from Algorithm 2 (possibly containing “ E ”s), we denote the set of non-empty bins $NonEmptyBin = \{k \in [K] : h_k(\mathbf{u}) \neq E\}$. The densification procedure scans over $k = 1, \dots, K$. For each k with $h_k(\mathbf{u}) = E$, we do:

1. Uniformly randomly pick a bin $k' \in NonEmptyBin$ that is non-empty.
2. (a) **OPH-fix:** we directly copy the k' -th hash value: $h_k(\mathbf{u}) \leftarrow h_{k'}(\mathbf{u})$.
- (b) **OPH-re:** we apply additional minwise hashing to bin $\mathcal{B}_{k'}$ using the “partial permutation” of $\mathcal{B}_k$ to get the hash for $h_k(\mathbf{u})$.

Specifically, In Algorithm 2, for re-randomized densification, $SortedIndex$ and $MapToIndex$ are used to define the within bin “partial permutation” $\pi^{(k)}$ of bin $\mathcal{B}_k$ for re-randomizing the empty bins.

It is shown that for both variants, the Jaccard estimator of the same form as (1) is unbiased. Li et al. (2019) showed that re-randomized densification always achieves smaller Jaccard estimation variance than that of fixed densification, and the improvement is especially significant when the data is sparse. Similar to b -bit MinHash, we can also keep the last b bits of the OPH hash values for convenient use.

3.2 DIFFERENTIAL PRIVATE ONE PERMUTATION HASHING (DP-OPH)

DP-OPH with densification. To privatize densified OPH, in Algorithm 4, we first take the last b bits of the hash values. Since the output space is finite with cardinality 2^b , we apply the randomized

Algorithm 4 Differentially Private Densified One Permutation Hashing (DP-OPH-fix, DP-OPH-re)**Input:** Densified OPH hash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$; number of bits b ; $\epsilon > 0, 0 < \delta < 1$ f : lower bound on the number of non-zeros in each data vector**Output:** b -bit DP-OPH values $\tilde{h}(\mathbf{u}) = [\tilde{h}_1(\mathbf{u}), \dots, \tilde{h}_K(\mathbf{u})]$

- 1: Take the last b bits of all hash values $\triangleright$ After which $h_k(\mathbf{u}) \in \{0, \dots, 2^b - 1\}$
- 2: Set $N = F_{fix}^{-1}(1 - \delta; D, K, f)$ (for DP-OPH-fix) or $N = F_{re}^{-1}(1 - \delta; D, K, f)$ (for DP-OPH-re),
and let $\epsilon' = \epsilon/N$
- 3: **for** $k = 1$ to K **do**
- 4: $\tilde{h}_k(\mathbf{u}) = \begin{cases} h_k(\mathbf{u}), & \text{with prob. } \frac{e^{\epsilon'}}{e^{\epsilon'} + 2^b - 1} \\ i, & \text{with prob. } \frac{1}{e^{\epsilon'} + 2^b - 1}, \text{ for } i \in \{0, \dots, 2^b - 1\}, i \neq h_k(\mathbf{u}) \end{cases}$
- 5: **end for**

response technique (Dwork & Roth, 2014; Wang et al., 2017) to flip the bits to achieve DP. After running Algorithm 3, suppose a densified OPH hash value $h_k(\mathbf{u}) = j, j \in 0, \dots, 2^b - 1$. With some $\epsilon' > 0$ that will be specified later, we output $\tilde{h}_k(\mathbf{u}) = j$ with probability $\frac{e^{\epsilon'}}{e^{\epsilon'} + 2^b - 1}$, and $\tilde{h}_k(\mathbf{u}) = i$ for $i \neq j$ with probability $\frac{1}{e^{\epsilon'} + 2^b - 1}$. It is easy to verify that, for a neighboring data $\mathbf{u}'$, when $h_k(\mathbf{u}') = j$, for $\forall i \in 0, \dots, 2^b - 1$, we have $\frac{P(\tilde{h}_k(\mathbf{u})=i)}{P(\tilde{h}_k(\mathbf{u}')=i)} = 1$; when $h_k(\mathbf{u}') \neq j$, we have $e^{-\epsilon'} \leq \frac{P(\tilde{h}_k(\mathbf{u})=i)}{P(\tilde{h}_k(\mathbf{u}')=i)} \leq e^{\epsilon'}$. Therefore, for a single hash value, this bit flipping satisfies ϵ' -DP.

It remains to determine ϵ' . Naively, since the perturbations (flipping) of the hash values are independent, by the composition property of DP (Dwork et al., 2006a), simply setting $\epsilon' = \epsilon/K$ for all K MinHash values would achieve overall ϵ -DP (for the hashed vector). However, since K is usually around hundreds, a very large ϵ value is required for this strategy to be useful. To this end, we can trade a small δ in the DP definition for a significantly reduced ϵ . Note that, not all the K hashed bits will change after we switch from $\mathbf{u}$ to its neighbor $\mathbf{u}'$. Assume each data vector contains at least f non-zeros, which is realistic since many data in practice have both high dimensionality D as well as many non-zero elements. Intuitively, when the data is not too sparse, $\mathbf{u}$ and $\mathbf{u}'$ tends to be similar. Therefore, the number of different hash values from Algorithm 3, $X = \sum_{k=1}^K \mathbb{1}\{h_k(\mathbf{u}) \neq h_k(\mathbf{u}')\}$, can be upper bounded by some N with probability $1 - \delta$. In the proof, this allows us to set $\epsilon' = \epsilon/N$ in the flipping probability and count δ as the failure probability in (ϵ, δ) -DP.

Next, we derive the distribution of X . Accordingly, in Algorithm 4, we set $N = F_{fix}^{-1}(1 - \delta; D, f, K)$ for DP-OPH-fix, $N = F_{re}^{-1}(1 - \delta; D, f, K)$ for DP-OPH-re, where $F_{fix}(x) = P(X \leq x)$ is the cumulative mass function (CMF) of X with OPH-fix ((2) + (3)), and F_{re} is the CMF of X with OPH-re ((2) + (4)), and F^{-1} is the inverse CMF. The proof can be found in Appendix B.

Lemma 3.1. Let $\mathbf{u}, \mathbf{u}' \in \{0, 1\}^D$ be neighbors. Denote $X = \sum_{k=1}^K \mathbb{1}\{h_k(\mathbf{u}) \neq h_k(\mathbf{u}')\}$ where the hashes are generated by Algorithm 3. Denote $f = |\mathbf{u}|$, $d = D/K$. We have

$$P(X = x) = \sum_{j=\max(0, K-f)}^{K-\lceil f/d \rceil} \sum_{z=1}^{\min(f, d)} \Theta(x, j, z|K), \quad \text{for } x = 0, \dots, K - \lceil f/d \rceil, \quad (2)$$

with $\Theta(x, j, z|K) = \tilde{P}(x|z, j)P(\tilde{f} = z|K - j)P(N_{emp} = j)$, where $P(\tilde{f} = z|K - j)$ is given in Lemma B.2, and $P(N_{emp} = j)$ is from Lemma B.1. Moreover,

$$\text{For OPH-fix: } \tilde{P}(x|z, j) = \mathbb{1}\{x = 0\}(1 - P_{\neq}) + \mathbb{1}\{x > 0\}P_{\neq} \cdot g_{bino}\left(x - 1; \frac{1}{K - j}, j\right), \quad (3)$$

$$\text{For OPH-re: } \tilde{P}(x|z, j) = (1 - P_{\neq}) \cdot g_{bino}\left(x; \frac{P_{\neq}}{K - j}, j\right) + P_{\neq} \cdot g_{bino}\left(x - 1; \frac{P_{\neq}}{K - j}, j\right), \quad (4)$$

where $g_{bino}(x; p, n)$ is the CMF of Binomial(p, n), and $P_{\neq}(z, b) = (1 - \frac{1}{2^b}) \frac{1}{z}$.

The privacy guarantee of DP-OPH with densification is shown as below.

Theorem 3.2. Both DP-OPH-fix and DP-OPH-re in Algorithm 4 achieve (ϵ, δ) -DP.

Algorithm 5 Differentially Private One Permutation Hashing with Random Bits (DP-OPH-rand)**Input:** OPH hash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$ from Algorithm 2; number of bits b ; $\epsilon > 0$ **Output:** DP-OPH-rand hash values $\tilde{h}(\mathbf{u}) = [\tilde{h}_1(\mathbf{u}), \dots, \tilde{h}_K(\mathbf{u})]$

- 1: Take the last b bits of all hash values ▷ After which $h_k(\mathbf{u}) \in \{0, \dots, 2^b - 1\}$
- 2: **for** $k = 1$ to K **do**
- 3: **if** $h_k(\mathbf{u}) \neq E$ **then**
- 4: $\tilde{h}_k(\mathbf{u}) = \begin{cases} h_k(\mathbf{u}), & \text{with prob. } \frac{e^\epsilon}{e^\epsilon + 2^b - 1} \\ i, & \text{with prob. } \frac{1}{e^{\epsilon'} + 2^b - 1}, \text{ for } i \in \{0, \dots, 2^b - 1\}, i \neq h_k(\mathbf{u}) \end{cases}$
- 5: **else**
- 6: $\tilde{h}_k(\mathbf{u}) = i$ with probability $\frac{1}{2^b}$, for $i = 0, \dots, 2^b - 1$ ▷ Assign random bits to empty bin
- 7: **end if**
- 8: **end for**

Algorithm 6 Differentially Private MinHash (DP-MH)**Input:** MinHash values $h_1(\mathbf{u}), \dots, h_K(\mathbf{u})$; number of bits b ; $\epsilon > 0, 0 < \delta < 1$ f : lower bound on the number of non-zeros in each data vector**Output:** DP-MH values $\tilde{h}(\mathbf{u}) = [\tilde{h}_1(\mathbf{u}), \dots, \tilde{h}_K(\mathbf{u})]$

- 1: Take the last b bits of all hash values ▷ After which $h_k(\mathbf{u}) \in \{0, \dots, 2^b - 1\}$
- 2: Set $N = F_{\text{bino}}^{-1}(1 - \delta; \frac{1}{f}, K)$, and $\epsilon' = \epsilon/N$
- 3: **for** $k = 1$ to K **do**
- 4: $\tilde{h}_k(\mathbf{u}) = \begin{cases} h_k(\mathbf{u}), & \text{with prob. } \frac{e^{\epsilon'}}{e^{\epsilon'} + 2^b - 1} \\ i, & \text{with prob. } \frac{1}{e^{\epsilon'} + 2^b - 1}, \text{ for } i \in \{0, \dots, 2^b - 1\}, i \neq h_k(\mathbf{u}) \end{cases}$
- 5: **end for**

DP-OPH without densification. From the practical perspective, we may also choose to privatize the OPH without densification (i.e., add DP to the output of Algorithm 2). The first step is to take the last b bits of every non-empty hash and get K hash values from $\{0, \dots, 2^b - 1\} \cup \{E\}$. Then, for non-empty bins, we keep the hash value with probability $\frac{e^\epsilon}{e^\epsilon + 2^b - 1}$, and randomly flip it otherwise. For empty bins (i.e., $h_k(\mathbf{u}) = E$), we simply assign a random value in $\{0, \dots, 2^b - 1\}$ to $\tilde{h}_k(\mathbf{u})$. The formal procedure of this so-called DP-OPH-rand method is summarized in Algorithm 5.

Theorem 3.3. *Algorithm 5 achieves ϵ -DP.*

Compared with Algorithm 4, DP-OPH-rand achieves strict DP with smaller flipping probability (effectively, $N \equiv 1$ in Algorithm 4). This demonstrates the essential benefit of the binning operation in OPH, since the change in one data coordinate will only affect one hash value (if densification is not applied). As a result, the non-empty hash values are less perturbed in DP-OPH-rand than in DP-OPH-fix or DP-OPH-re. But this comes with an extra cost as we have to assign random bits to empty bins which do not provide any useful information, and this extra cost does not diminish as ϵ increases because the number of empty bins only depends on the data itself and K .

DP-MinHash. While we have presented our main DP-OPH algorithms, we also present a DP MinHash (DP-MH) method (Algorithm 6) as a baseline comparison. The mechanism of DP-MH is similar to that of densified DP-OPH. The difference between Algorithm 6 and Algorithm 4 is in the calculation of N . In Algorithm 6, we set $N = F_{\text{bino}}^{-1}(1 - \delta; \frac{1}{f}, K)$ where $F_{\text{bino}}^{-1}(x; p, n)$ is the inverse cumulative mass function of $\text{Binomial}(p, n)$ with n trials and success probability p .

Theorem 3.4. *Algorithm 6 is (ϵ, δ) -DP.*

The proof strategy is similar to Theorem 3.2 by noting that $X = \sum_{k=1}^K \mathbb{1}\{h_k(\mathbf{u}) \neq h_k(\mathbf{u}')\}$ for neighboring $\mathbf{u}$ and $\mathbf{u}'$ follows $\text{Binomial}(\frac{1}{f}, K)$. In a related work, Aumüller et al. (2020) also proposed to apply randomized response to MinHash. However, the authors incorrectly used a tail bound for the binomial distribution (see their Lemma 1) which is only valid for small deviation. In DP, δ is often very small (e.g., 10^{-6}), so the large deviation tail bound should be used which is

looser than the one used therein¹. That said, in their paper, the perturbation is underestimated and their method does not satisfy DP. In our Algorithm 6, we fix it by using the exact probability mass function to compute the tail probability, avoiding any loss due to the concentration bounds.

3.3 COMPARISON OF DP-OPH AND DP-MH

We first analyze the mean of the Jaccard estimators and derive unbiased estimators of J . To simplify the formula, we assume that $\mathbf{u}$ and $\mathbf{v}$ have the same “privacy discount factor” N (which implies that $\mathbf{u}$ and $\mathbf{v}$ have similar sparsity). The results can be easily extended to the general case.

Theorem 3.5. *For $u, v \in \{0, 1\}^D$, denote $f_u = |\mathbf{u}|$, $f_v = |\mathbf{v}|$, $a = |\mathbf{u} \cap \mathbf{v}|$. Suppose $\mathbf{u}$ and $\mathbf{v}$ have the same privacy discount factor N in Algorithm 4 or Algorithm 6. Then, $J = \frac{a}{f_u + f_v - a}$. Denote $p = \frac{\exp(\epsilon/N)}{\exp(\epsilon/N) + 2^b - 1}$. For DP-OPH-fix, DP-OPH-re, and DP-MH, define $\hat{J} = \frac{1}{K} \sum_{k=1}^K \mathbb{1}\{\tilde{h}_k(\mathbf{u}) = \tilde{h}_k(\mathbf{v})\}$. We have $\mathbb{E}[\hat{J}] = \frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b}$. Thus, an unbiased estimator is $\hat{J}_{unbias} = \frac{(2^b - 1)(2^b \hat{J} - 1)}{(2^b p - 1)^2}$.*

The variances of the unbiased estimators defined in Theorem 3.5 are given as below.

Theorem 3.6. *Define $J_B = J + (1 - J)\frac{1}{2^b}$, $\tilde{J} = \frac{a-1}{f_u + f_v - a - 1}$. Denote $c_1 = p^2 + \frac{(1-p)^2}{2^b - 1}$, and $c_2 = \frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2}(1 - p)^2$. Define $\zeta(m) = \mathbb{E}[\frac{1}{\tilde{f}} | m]$ where the conditional distribution of $\tilde{f}$ is given in Lemma B.2, and:*

$$\begin{aligned} \tau_{11} &= J\tilde{J}, \quad \tau_{10} = J - J\tilde{J}, \quad \tau_{00} = 1 - 2J + J\tilde{J}, \\ \tau_{11,f}(m) &= \frac{1}{m}J + \frac{m-1}{m}J\tilde{J}, \quad \tau_{10,f}(m) = \frac{m-1}{m}(J - J\tilde{J}), \quad \tau_{00,f}(m) = 1 - (2 - \frac{1}{m})J + \frac{m-1}{m}J\tilde{J}, \\ \tau_{11,r}(m) &= \frac{\zeta(m)}{m}J + \frac{m - \zeta(m)}{m}J\tilde{J}, \quad \tau_{10,r}(m) = \frac{m - \zeta(m)}{m}(J - J\tilde{J}), \\ \tau_{00,r}(m) &= 1 - (2 - \frac{\zeta(m)}{m})J + \frac{m - \zeta(m)}{m}J\tilde{J}. \end{aligned}$$

Further denote $P_{11} = \tau_{11} + \frac{1}{2^b - 1}\tau_{10} + \frac{1}{2^{2b}}\tau_{00}$, $P_{10} = J_B - P_{11}$, $P_{00} = 1 - 2J_B + P_{11}$, and $(P_{11,f}, P_{10,f}, P_{00,f})$ and $(P_{11,r}, P_{10,r}, P_{00,r})$ analogously by replacing $(\tau_{11}, \tau_{10}, \tau_{00})$ with $(\tau_{11,f}, \tau_{10,f}, \tau_{00,f})$ and $(\tau_{11,r}, \tau_{10,r}, \tau_{00,r})$, respectively. We have for DP-MH:

$$\text{Var}[\hat{J}_{unbias,MH}] = \frac{1}{K} \left(\frac{(2^b p + 1)^2}{(2^b p - 1)^2} J + \frac{2^b - 1}{(2^b p - 1)^2} \right) \left(\frac{(2^b - 1)^2}{(2^b p - 1)^2} - \frac{(2^b p + 1)^2}{(2^b p - 1)^2} J \right).$$

For DP-OPH: Let $m = K - N_{emp}$ where N_{emp} is distributed as Lemma B.1.

$$\text{Var}[\hat{J}_{unbias,OPH}] = \frac{2^{2b}(2^b - 1)^2}{(2^b p - 1)^4} \left[\frac{1}{K} \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right) + \frac{1}{K^2} A - \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right)^2 \right],$$

$$A = \mathbb{E}_m[m(m-1)H_N + (K-m)(K+m-1)H_E],$$

with $H_N = c_1^2 P_{11} + 2c_1 c_2 P_{10} + c_2^2 P_{00}$. For DP-OPH-fix, $H_E = c_1^2 P_{11,f} + 2c_1 c_2 P_{10,f} + c_2^2 P_{00,f}$; for DP-OPH-re, $H_E = c_1^2 P_{11,r} + 2c_1 c_2 P_{10,r} + c_2^2 P_{00,r}$.

Comparison: Densified DP-OPH vs. DP-MH. We show that OPH is a better method than MinHash from the privacy perspective. Firstly, we compare N , the privacy discount factor, in DP-OPH-fix, DP-OPH-re, and DP-MH. Smaller N leads to smaller bit flipping probability which benefits the utility. In Figure 1, we plot N vs. f , for $D = 1024$, $K = 64$, and $\delta = 10^{-6}$. Similar comparison also holds for other D, K combinations. We observe that N in DP-OPH is typically smaller than that in DP-MH. Moreover, N for DP-OPH-re is consistently smaller than that of DP-OPH-fix. This illustrates that re-randomization in densification is an important step to achieve stronger privacy.

In Figure 2, we plot the empirical MSE of the unbiased estimators. The data is simulated with $f_u = f_v = f$, and $a = f/2$ (see notations in Theorem 3.5). The empirical MSE matches the variances in Theorem 3.6. DP-OPH-re has smaller variance than DP-OPH-fix and DP-MH.

¹For X following a Binomial distribution with mean μ , Aumüller et al. (2020) used the concentration inequality $P(X \geq (1 + \xi)\mu) \leq \exp(-\frac{\xi^2 \mu}{3})$, which only holds when $0 \leq \xi \leq 1$. For large deviations (large ξ), the valid Binomial tail bound should be $P(X \geq (1 + \xi)\mu) \leq \exp(-\frac{\xi^2 \mu}{\xi + 2})$.

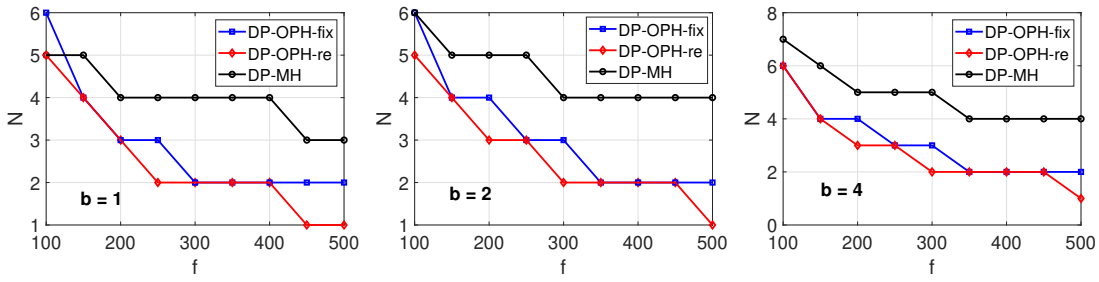


Figure 1: Comparison of the privacy discount factor N for densified DP-OPH and DP-MH, against the number of non-zero elements in the data vector f . $D = 1024$, $K = 64$, $\delta = 10^{-6}$.

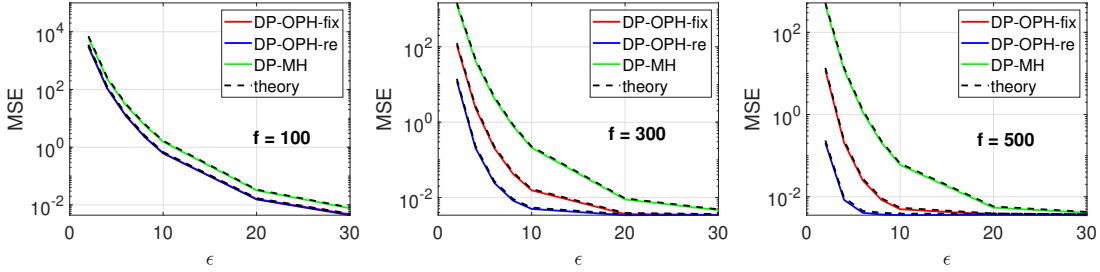


Figure 2: MSE comparison of the unbiased Jaccard estimators (Theorem 3.5). The dash curves are theoretical variances in Theorem 3.6. $D = 1024$, $K = 64$, $\delta = 10^{-6}$. $b = 4$.

4 EXPERIMENTS

We conduct similarity search on two datasets from genome and web where MinHash-type algorithms are widely used: (1) the Leukemia gene expression dataset (<https://sbcb.inf.ufrgs.br/cumida>); (2) the Webspam (Chang & Lin, 2011) dataset for spam detection. Both datasets are binarized to 0/1. For Leukemia, we first standardize the features columns (to mean 0 and std 1), and then keep entries larger than 1 to be 1 and zero out the others. For Webspam, the entries are non-negative and we simply set the positive elements to 1. For Leukemia, we treat each data point as the query and other points as the database for search. For Webspam, we use the training set as the database, and the test set as queries. For each query point, we set the ground truth (“gold-standard”) neighbors as the top 50 data points in the database with the highest Jaccard similarities to the query.

Setup. To search with DP-OPH and DP-MH, we generate the private hash values and compute the collision estimator between the query and each data point. Then, we retrieve the data points with the highest estimated Jaccard similarities to the query. For densified DP-OPH (Algorithm 4) and DP-MH (Algorithm 6), we ensure the lower bound f on the number of non-zero elements by filtering the data points with at least f non-zeros. We use $f = 1000, 500$ for Leukemia and Webspam, respectively, which cover 100% and 90% of the total data points.

Results. In Figure 3, we report the precision for Leukemia with $b = 1, 2, 4$ and $\epsilon \in [1, 30]$. The ϵ range is common in the literature of DP hashing, e.g., the $[2.45, 33.5]$ reported in Zhao et al. (2022) which studied private count-sketch. The recall comparisons are similar. The results are averaged over all query points and over 5 runs. We observe that:

- DP-OPH-re outperforms DP-MH and DP-OPH-fix, at all ϵ levels. That is, DP-OPH-re is a uniformly more superior method than the existing DP-MH method for private hashing.
- DP-OPH-rand achieves good accuracy with small ϵ (e.g., $\epsilon < 5$), but stops improving with ϵ afterwards (due to the random bits for the empty bins), justifying the trade-off discussed in Section 3.2. When ϵ gets larger (e.g., $\epsilon = 5 \sim 15$), DP-OPH-re performs the best.

The results on Webspam are presented in Figure 4. Similarly, DP-OPH-re achieves better performance than DP-MH and DP-OPH-fix for all ϵ . DP-OPH-rand performs the best with $\epsilon < 10$. DP-OPH-re bypasses DP-OPH-rand as ϵ grows larger.

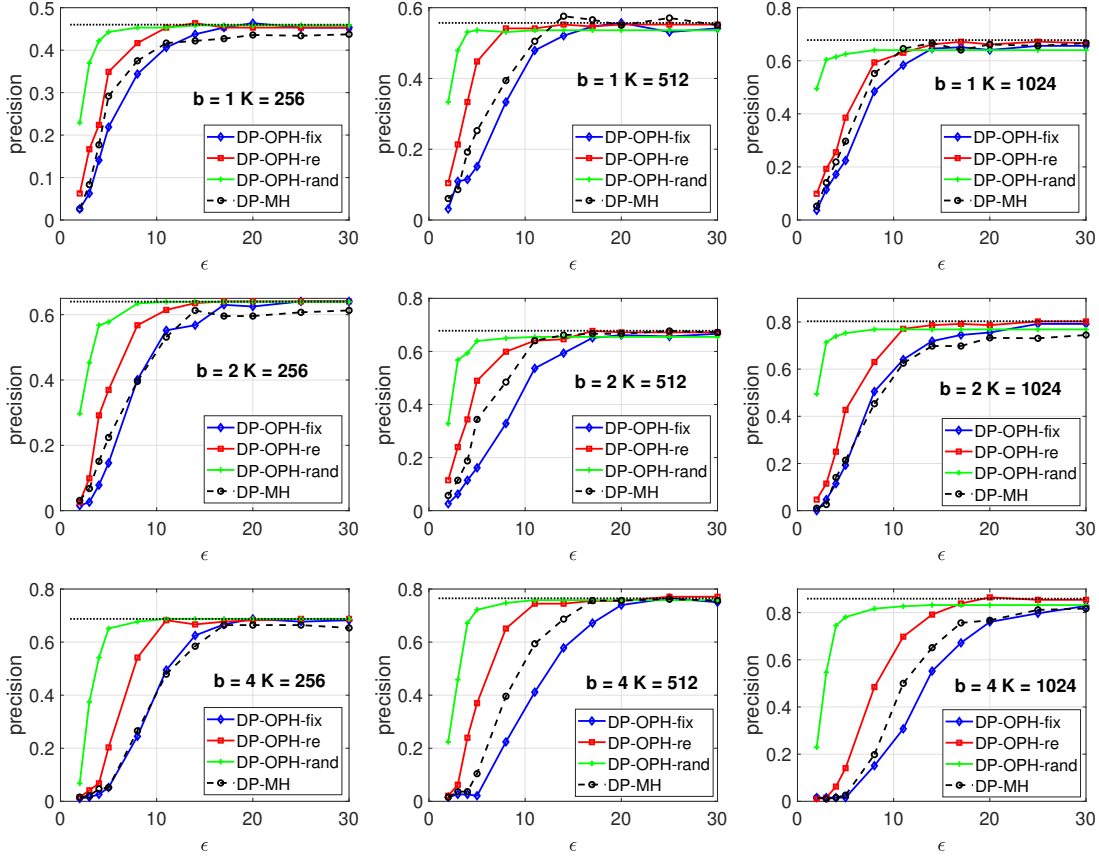


Figure 3: Precision@1 results on Leukemia gene expression dataset with $b = 1, 2, 4$. $\delta = 10^{-6}$. Dotted curves are for non-private OPH-re.

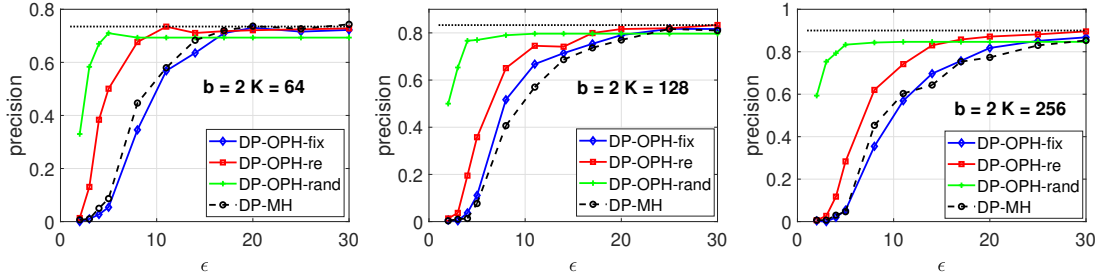


Figure 4: Precision@10 results on Webspam dataset with $b = 2$. $\delta = 10^{-6}$.

5 CONCLUSION

In this paper, we study differentially privatized one permutation hashing (DP-OPH) methods. We develop three variants depending on the densification procedure of OPH, and provide privacy and utility analyses of our algorithms. We show the significant advantages of our DP-OPH over the DP MinHash alternative proposed in prior literature for hashing the Jaccard similarity at various privacy levels. Experiments are conducted on retrieval tasks to justify the effectiveness of the proposed DP-OPH, and provide guidance on the appropriate choice of the DP-OPH variant in different scenarios. In Appendix A, we also provide DP-BCWS which is based on bin-wise consistent weighted samples (BCWS) (Li et al., 2019) for weighted Jaccard similarity (for non-negative data). Given the efficiency and good performance, we expect DP-OPH to serve as a useful privatized alternative in practical applications where MinHash-type methods are heavily used. In the appendix, we also provide an extension of DP-OPH to real-value data called DP-BCWS.

REFERENCES

- Martín Abadi, Andy Chu, Ian J. Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 308–318, Vienna, Austria, 2016.
- Naman Agarwal, Ananda Theertha Suresh, Felix X. Yu, Sanjiv Kumar, and Brendan McMahan. cpSGD: Communication-efficient and differentially-private distributed SGD. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7575–7586, Montréal, Canada, 2018.
- A. Asuncion and D.J. Newman. UCI machine learning repository, 2007. URL [http://www.ics.uci.edu/\\$sim\\$mllearn/{MLR}epository.html](http://www.ics.uci.edu/simmllearn/{MLR}epository.html).
- Martin Aumüller, Anders Bourgeat, and Jana Schmur. Differentially private sketches for jaccard similarity estimation. *CoRR*, abs/2008.08134, 2020.
- Michael Bendersky and W. Bruce Croft. Finding text reuse on the web. In *Proceedings of the Second International Conference on Web Search and Web Data Mining (WSDM)*, pp. 262–271, Barcelona, Spain, 2009.
- Konstantin Berlin, Sergey Koren, Chen-Shan Chin, James P Drake, Jane M Landolin, and Adam M Phillippy. Assembling large genomes with single-molecule sequencing and locality-sensitive hashing. *Nature biotechnology*, 33(6):623–630, 2015.
- Jeremiah Blocki, Avrim Blum, Anupam Datta, and Or Sheffet. The johnson-lindenstrauss transform itself preserves differential privacy. In *Proceedings of the 53rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 410–419, New Brunswick, NJ, 2012.
- Avrim Blum, Cynthia Dwork, Frank McSherry, and Kobbi Nissim. Practical privacy: the SuLQ framework. In *Proceedings of the Twenty-fourth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS)*, pp. 128–138, Baltimore, MD, 2005.
- Andrei Z Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences (SEQUENCES)*, pp. 21–29, Salerno, Italy, 1997.
- Andrei Z. Broder, Steven C. Glassman, Mark S. Manasse, and Geoffrey Zweig. Syntactic clustering of the web. *Comput. Networks*, 29(8-13):1157–1166, 1997.
- C. Titus Brown and Luiz Irber. sourmash: a library for minhash sketching of DNA. *J. Open Source Softw.*, 1(5):27, 2016.
- Gregory Buehrer and Kumar Chellapilla. A scalable pattern mining approach to web graph compression with communities. In *Proceedings of the International Conference on Web Search and Web Data Mining (WSDM)*, pp. 95–106, Stanford, CA, 2008.
- Chih-Chung Chang and Chih-Jen Lin. Libsvm: a library for support vector machines. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 2(3):27, 2011.
- Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. *Theor. Comput. Sci.*, 312(1):3–15, 2004.
- Moses S Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing (STOC)*, pp. 380–388, Montreal, Canada, 2002.
- Kamalika Chaudhuri and Claire Monteleoni. Privacy-preserving logistic regression. In *Advances in Neural Information Processing Systems (NIPS)*, pp. 289–296, Vancouver, Canada, 2008.
- Kamalika Chaudhuri, Claire Monteleoni, and Anand D. Sarwate. Differentially private empirical risk minimization. *J. Mach. Learn. Res.*, 12:1069–1109, 2011.

- Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Michael Mitzenmacher, Alessandro Panconesi, and Prabhakar Raghavan. On compressing social networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 219–228, Paris, France, 2009.
- Ondrej Chum and Jiri Matas. Fast computation of min-hash signatures for image collections. In *Proceedings of the 2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3077–3084, Providence, RI, 2012.
- Søren Dahlgaard, Mathias Bæk Tejs Knudsen, and Mikkel Thorup. Practical hash functions for similarity estimation and dimensionality reduction. In *Advances in Neural Information Processing Systems (NIPS)*, pp. 6615–6625, Long Beach, CA, USA, 2017.
- Abhinandan Das, Mayur Datar, Ashutosh Garg, and Shyamsundar Rajaram. Google news personalization: scalable online collaborative filtering. In *Proceedings of the 16th International Conference on World Wide Web (WWW)*, pp. 271–280, Banff, Alberta, Canada, 2007.
- Fan Deng, Stefan Siersdorfer, and Sergej Zerr. Efficient jaccard-based diversity analysis of large document collections. In *Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM)*, pp. 1402–1411, Maui, HI, 2012.
- Charlie Dickens, Justin Thaler, and Daniel Ting. Order-invariant cardinality estimators are differentially private. In *Advances in Neural Information Processing Systems (NeurIPS)*, New Orleans, LA, 2022.
- Jinshuo Dong, Aaron Roth, and Weijie J Su. Gaussian differential privacy. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 84(1):3–37, 2022.
- Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3-4):211–407, 2014.
- Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: Privacy via distributed noise generation. In *Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings*, volume 4004 of *Lecture Notes in Computer Science*, pp. 486–503. Springer, 2006a.
- Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam D. Smith. Calibrating noise to sensitivity in private data analysis. In *Proceedings of the Third Theory of Cryptography Conference (TCC)*, pp. 265–284, New York, NY, 2006b.
- Dan Feldman, Amos Fiat, Haim Kaplan, and Kobbi Nissim. Private coresets. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pp. 361–370, Bethesda, MD, 2009.
- Weiqi Feng and Dong Deng. Allign: Aligning all-pair near-duplicate passages in long texts. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pp. 541–553, Virtual Event, China, 2021.
- Dennis Fetterly, Mark Manasse, Marc Najork, and Janet L. Wiener. A large-scale study of the evolution of web pages. In *Proceedings of the Twelfth International World Wide Web Conference (WWW)*, pp. 669–678, Budapest, Hungary, 2003.
- Philippe Flajolet and G. Nigel Martin. Probabilistic counting algorithms for data base applications. *J. Comput. Syst. Sci.*, 31(2):182–209, 1985.
- Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. In *Discrete Mathematics and Theoretical Computer Science*, pp. 137–156, 2007.
- Jason Ge, Zhaoran Wang, Mengdi Wang, and Han Liu. Minimax-optimal privacy-preserving sparse PCA in distributed systems. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 1589–1598, Playa Blanca, Lanzarote, Canary Islands, Spain, 2018.

- Anupam Gupta, Katrina Ligett, Frank McSherry, Aaron Roth, and Kunal Talwar. Differentially private combinatorial optimization. In *Proceedings of the Twenty-First Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1106–1125, Austin, TX, 2010.
- Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing (STOC)*, pp. 604–613, Dallas, TX, 1998.
- Sergey Ioffe. Improved consistent sampling, weighted minhash and L1 sketching. In *Proceedings of the 10th IEEE International Conference on Data Mining (ICDM)*, pp. 246–255, Sydney, Australia, 2010.
- Peng Jia, Pinghui Wang, Junzhou Zhao, Shuo Zhang, Yiyang Qi, Min Hu, Chao Deng, and Xiaohong Guan. Bidirectionally densifying LSH sketches with empty bins. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pp. 830–842, Virtual Event, China, 2021.
- Nan Jiang, Chen Luo, Vihan Lakshman, Yesh Dattatreya, and Yexiang Xue. Massive text normalization via an efficient randomized algorithm. In *WWW ’22: The ACM Web Conference 2022, Virtual Event, Lyon, France, April 25 - 29, 2022*, pp. 2946–2956. ACM, 2022.
- Shiva Prasad Kasiviswanathan, Kobbi Nissim, Sofya Raskhodnikova, and Adam D. Smith. Analyzing graphs with node differential privacy. In *Proceedings of the 10th Theory of Cryptography Conference, TCC*, volume 7785, pp. 457–476, Tokyo, Japan, 2013.
- Krishnamurthy Kenthapadi, Aleksandra Korolova, Ilya Mironov, and Nina Mishra. Privacy via the johnson-lindenstrauss transform. *J. Priv. Confidentiality*, 5(1), 2013.
- David C. Lee, Qifa Ke, and Michael Isard. Partition min-hash for partial duplicate image discovery. In *Proceedings of the 11th European Conference on Computer Vision (ECCV), Part I*, pp. 648–662, Heraklion, Crete, Greece, 2010.
- Jakub Lemiesz. On the algebra of data sketches. *Proc. VLDB Endow.*, 14(9):1655–1667, 2021.
- Jin Li, Sudipta Sengupta, Ran Kalach, Ronakkumar N Desai, Paul Adrian Oltean, and James Robert Benton. Using index partitioning and reconciliation for data deduplication, August 18 2015. US Patent 9,110,936.
- Ping Li and Arnd Christian König. b-bit minwise hashing. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pp. 671–680, Raleigh, NC, 2010.
- Ping Li, Anshumali Shrivastava, Joshua L. Moore, and Arnd Christian König. Hashing algorithms for large-scale learning. In *Advances in Neural Information Processing Systems (NIPS)*, pp. 2672–2680, Granada, Spain, 2011.
- Ping Li, Art B Owen, and Cun-Hui Zhang. One permutation hashing. In *Advances in Neural Information Processing Systems (NIPS)*, pp. 3122–3130, Lake Tahoe, NV, 2012.
- Ping Li, Xiaoyun Li, and Cun-Hui Zhang. Re-randomized densification for one permutation hashing and bin-wise consistent weighted sampling. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 15900–15910, Vancouver, Canada, 2019.
- Ping Li, Xiaoyun Li, Gennady Samorodnitsky, and Weijie Zhao. Consistent sampling through extremal process. In *Proceedings of the Web Conference (WWW)*, pp. 1317–1327, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021, 2021.
- Xiaoyun Li and Ping Li. C-MinHash: Improving minwise hashing with circulant permutation. In *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 12857–12887, Baltimore, MD, 2022.
- Mark Manasse, Frank McSherry, and Kunal Talwar. Consistent weighted sampling. Technical Report MSR-TR-2010-73, Microsoft Research, 2010.
- Fatemeh Nargesian, Erkang Zhu, Ken Q. Pu, and Renée J. Miller. Table union search on open data. *Proc. VLDB Endow.*, 11(7):813–825, 2018.

- Brian D Ondov, Todd J Treangen, Páll Melsted, Adam B Mallonee, Nicholas H Bergman, Sergey Koren, and Adam M Phillippy. Mash: fast genome and metagenome distance estimation using minhash. *Genome biology*, 17(1):1–14, 2016.
- Sandeep Pandey, Andrei Broder, Flavio Chierichetti, Vanja Josifovski, Ravi Kumar, and Sergei Vasilvitskii. Nearest-neighbor caching for content-match applications. In *Proceedings of the 18th International Conference on World Wide Web (WWW)*, pp. 441–450, Madrid, Spain, 2009.
- Rajen Dinesh Shah and Nicolai Meinshausen. On b-bit min-wise hashing for large-scale regression and classification with sparse data. *J. Mach. Learn. Res.*, 18:178:1–178:42, 2017.
- Anshumali Shrivastava. Optimal densification for fast and accurate minwise hashing. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 3154–3163, Sydney, Australia, 2017.
- Anshumali Shrivastava and Ping Li. In defense of minhash over simhash. In *Proceedings of the Seventeenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 886–894, Reykjavik, Iceland, 2014.
- Adam D. Smith, Shuang Song, and Abhradeep Thakurta. The flajolet-martin sketch itself preserves differential privacy: Private counting with minimal space. In *Advances in Neural Information Processing Systems*, virtual, 2020.
- Nina Mesing Stausholm. Improved differentially private euclidean distance approximation. In *Proceedings of the 40th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, pp. 42–56, Virtual Event, China, 2021.
- Acar Tamersoy, Kevin A. Roundy, and Duen Horng Chau. Guilt by association: large scale malware detection by mining file-relation graphs. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1524–1533, New York, NY, 2014.
- Tom Tseng, Laxman Dhulipala, and Julian Shun. Parallel index-based structural graph clustering and its approximation. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pp. 1851–1864, Virtual Event, China, 2021.
- Santosh S Vempala. *The random projection method*, volume 65. American Mathematical Soc., 2005.
- Pinghui Wang, Yiyan Qi, Yuanming Zhang, Qiaozhu Zhai, Chenxu Wang, John C. S. Lui, and Xiaohong Guan. A memory-efficient sketch method for estimating high similarities in streaming sets. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pp. 25–33, Anchorage, AK, 2019.
- Tianhao Wang, Jeremiah Blocki, Ninghui Li, and Somesh Jha. Locally differentially private protocols for frequency estimation. In *Proceedings of the 26th USENIX Security Symposium, USENIX Security (USENIX)*, pp. 729–745, Vancouver, Canada, 2017.
- Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H. Yang, Farhad Farokhi, Shi Jin, Tony Q. S. Quek, and H. Vincent Poor. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE Trans. Inf. Forensics Secur.*, 15:3454–3469, 2020.
- Jia Xu, Zhenjie Zhang, Xiaokui Xiao, Yin Yang, Ge Yu, and Marianne Winslett. Differentially private histogram publication. *VLDB J.*, 22(6):797–822, 2013.
- Yun William Yu and Griffin M. Weber. Hyperminhash: Minhash in loglog space. *IEEE Trans. Knowl. Data Eng.*, 34(1):328–339, 2022.
- Jun Zhang, Zhenjie Zhang, Xiaokui Xiao, Yin Yang, and Marianne Winslett. Functional mechanism: Regression analysis under differential privacy. *Proc. VLDB Endow.*, 5(11):1364–1375, 2012.
- Fuheng Zhao, Dan Qiao, Rachel Redberg, Divyakant Agrawal, Amr El Abbadi, and Yu-Xiang Wang. Differentially private linear sketches: Efficient implementations and applications. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Weijie Zhao, Deping Xie, Ronglai Jia, Yulei Qian, Ruiquan Ding, Mingming Sun, and Ping Li. Distributed hierarchical GPU parameter server for massive scale deep learning ads systems. In *Proceedings of Machine Learning and Systems 2020 (MLSys)*, Austin, TX, 2020.

Erkang Zhu, Ken Q. Pu, Fatemeh Nargesian, and Renée J. Miller. Interactive navigation of open data linkages. *Proc. VLDB Endow.*, 10(12):1837–1840, 2017.

A EXTENSION: DIFFERENTIALLY PRIVATE BIN-WISE CONSISTENT WEIGHTED SAMPLING (DP-BCWS) FOR WEIGHTED JACCARD SIMILARITY

Algorithm 7 Consistent Weighted Sampling (CWS)

Input: Non-negative data vector $\mathbf{u} \in \mathbb{R}_+^D$
Output: Consistent weighted sampling hash $h^* = (i^*, t^*)$

- 1: **for** every non-zero v_i **do**
- 2: $r_i \sim \text{Gamma}(2, 1)$, $c_i \sim \text{Gamma}(2, 1)$, $\beta_i \sim \text{Uniform}(0, 1)$
- 3: $t_i \leftarrow \lfloor \frac{\log u_i}{r_i} + \beta_i \rfloor$, $y_i \leftarrow \exp(r_i(t_i - \beta_i))$
- 4: $a_i \leftarrow c_i / (y_i \exp(r_i))$
- 5: **end for**
- 6: $i^* \leftarrow \arg \min_i a_i$, $t^* \leftarrow t_{i^*}$

In our main paper, we focused on DP hashing algorithms for the binary Jaccard similarity. Indeed, our algorithm can also be extended to hashing the weighted Jaccard similarity: (recall the definition)

$$J_w(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^D \min\{u_i, v_i\}}{\sum_{i=1}^D \max\{u_i, v_i\}}, \quad (5)$$

for two non-negative data vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}_+$. The standard hashing algorithm for (5) is called Consistent Weighted Sampling (CWS) as summarized in Algorithm 7 (Ioffe, 2010; Manasse et al., 2010; Li et al., 2021). To generate one hash value, we need three length- D random vectors $\mathbf{r} \sim \text{Gamma}(2, 1)$, $\mathbf{c} \sim \text{Gamma}(2, 1)$ and $\boldsymbol{\beta} \sim \text{Uniform}(0, 1)$. We denote Algorithm 7 as a function $CWS(\mathbf{u}; \mathbf{r}, \mathbf{c}, \boldsymbol{\beta})$. Li et al. (2019) proposed bin-wise CWS (BCWS) which exploits the same idea of binning as in OPH. The binning and densification procedure of BCWS is exactly the same as OPH (Algorithm 2 and Algorithm 3), except that every time we apply CWS, instead of MinHash, to the data in the bins to generate hash values. Note that in CWS, the output contains two values: i^* is a location index similar to the output of OPH, and t^* is a real-value scalar. Prior studies (e.g., Li et al. (2021)) showed that the second element has minimal impact on the estimation accuracy in most practical cases (i.e., only counting the collision of the first element suffices). Therefore, in our study, we only keep the first integer element as the hash output for subsequent learning tasks.

For weighted data vectors, we follow the prior DP literature on weighted sets (e.g., Xu et al. (2013); Smith et al. (2020); Dickens et al. (2022); Zhao et al. (2022)) and define the neighboring data vectors as those who differ in one element. To privatize BCWS, there are also three possible ways depending on the densification option. Since the DP algorithm design for densified BCWS requires rigorous and non-trivial computations which might be an independent study, here we empirically test the (b -bit) DP-BCWS method with random bits for empty bins. The details are provided in Algorithm 8. In general, we first randomly split the data entries into K equal length bins, and apply CWS to the data $\mathbf{u}_{\mathcal{B}_k}$ in each non-empty bin $\mathcal{B}_k$ using the random numbers $(\mathbf{r}_{\mathcal{B}_k}, \mathbf{c}_{\mathcal{B}_k}, \boldsymbol{\beta}_{\mathcal{B}_k})$ to generate K hash values (possibly including empty bins). After each hash is truncated to b bits, we uniformly randomly assign a hash value in $\{0, \dots, 2^b - 1\}$ to every empty bin.

Using the same proof arguments as Theorem 3.3, we have the following guarantee.

Theorem A.1. *Algorithm 8 satisfies ϵ -DP.*

Empirical evaluation. In Figure 5, we train an l_2 -regularized logistic regression on the DailySports dataset², and report the test accuracy with various b and K values. The l_2 regularization parameter λ is tuned over a fine grid from 10^{-4} to 10. Similar to the results in the previous section, the performance of DP-BCWS becomes stable as long as $\epsilon > 5$. Note that, linear logistic regression only gives $\approx 75\%$ accuracy on original DailySports dataset (without DP). With DP-BCWS, the accuracy can reach $\approx 98\%$ with $K = 1024$ and $\epsilon = 5$.

²<https://archive.ics.uci.edu/ml/datasets/daily+and+sports+activities>

Algorithm 8 Differential Private Bin-wise Consistent Weighted Sampling (DP-BCWS)**Input:** Binary vector $\mathbf{u} \in \{0, 1\}^D$; number of hash values K ; number of bits per hash b **Output:** DP-BCWS hash values $\tilde{h}_1(\mathbf{u}), \dots, \tilde{h}_K(\mathbf{u})$

```

1: Generate length- $D$  random vectors  $\mathbf{r} \sim \text{Gamma}(2, 1)$ ,  $\mathbf{c} \sim \text{Gamma}(2, 1)$ ,  $\beta \sim \text{Uniform}(0, 1)$ 
2: Let  $d = D/K$ . Use a permutation  $\pi : [D] \mapsto [D]$  with fixed seed to randomly split  $[D]$  into  $K$ 
   equal-size bins  $\mathcal{B}_1, \dots, \mathcal{B}_K$ , with  $\mathcal{B}_k = \{j \in [D] : (k-1)d + 1 \leq \pi(j) \leq kd\}$ 
3: for  $k = 1$  to  $K$  do
4:   if Bin  $\mathcal{B}_k$  is non-empty then
5:      $h_k(\mathbf{u}) \leftarrow \text{CWS}(\mathbf{u}_{\mathcal{B}_k}; \mathbf{r}_{\mathcal{B}_k}, \mathbf{c}_{\mathcal{B}_k}, \beta_{\mathcal{B}_k})$  ▷ Run CWS within each non-empty bin
6:      $h_k(\mathbf{u}) \leftarrow$  last  $b$  bits of  $h_k(\mathbf{u})$ 
7:      $\tilde{h}_k(\mathbf{u}) = \begin{cases} h_k(\mathbf{u}), & \text{with probability } \frac{e^\epsilon}{e^\epsilon + 2^b - 1} \\ i, & \text{with probability } \frac{1}{e^\epsilon + 2^b - 1}, \text{ for } i \in \{0, \dots, 2^b - 1\}, i \neq h_k(\mathbf{u}) \end{cases}$ 
8:   else
9:      $h_k(\mathbf{u}) \leftarrow E$ 
10:     $\tilde{h}_k(\mathbf{u}) = i$  with probability  $\frac{1}{2^b}$ , for  $i = 0, \dots, 2^b - 1$  ▷ Assign random bits to empty bin
11:   end if
12: end for

```

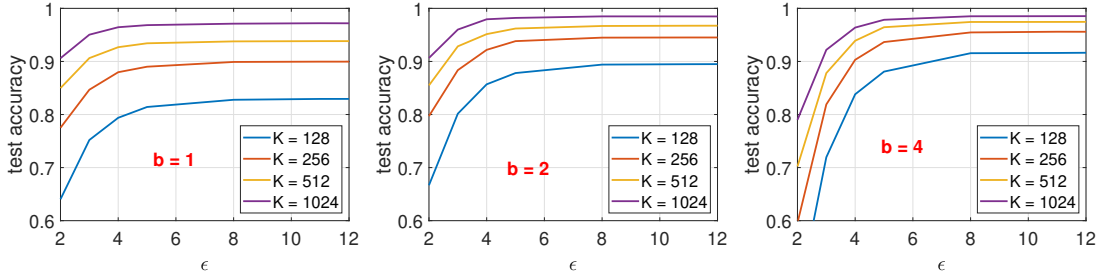


Figure 5: Test classification accuracy of DP-BCWS on DailySports dataset (Asuncion & Newman, 2007) with l_2 -regularized logistic regression.

In Figure 6, we train a neural network with two hidden layers of size 256 and 128 respectively on MNIST. We use the ReLU activation function and the standard cross-entropy loss. We see that, in a reasonable privacy regime (e.g., $\epsilon < 10$), DP-BCWS is able to achieve $\approx 95\%$ test accuracy with proper K and b combinations (one can choose the values depending on practical scenarios and needs). For example, with $b = 4$ and $K = 128$, DP-BCWS achieves $\approx 97\%$ accuracy at $\epsilon = 8$.

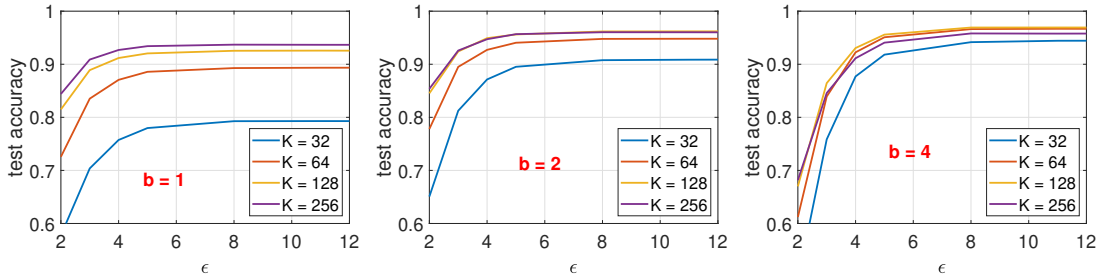


Figure 6: Test classification accuracy of DP-BCWS on MNIST with 2-hidden layer neural network.

B PROOFS

Lemma B.1 (Li et al. (2012)). Let $f = |\{i : u_i = 1\}|$, and $I_{emp,k}$ be the indicator function that the k -th bin is empty, and $N_{emp} = \sum_{k=1}^K I_{emp,k}$. Suppose $\text{mod}(D, K) = 0$. We have

$$P(N_{emp} = j) = \sum_{\ell=0}^{K-j} (-1)^\ell \binom{K}{j} \binom{K-j}{\ell} \binom{D(1-(j+\ell)/K)}{f} / \binom{D}{f}.$$

Lemma B.2 (Li et al. (2019)). Conditional on the event that m bins are non-empty, let $\tilde{f}$ be the number of non-zero elements in a non-empty bin. Denote $d = D/K$. The conditional probability distribution of $\tilde{f}$ is given by

$$P(\tilde{f} = j | m) = \frac{\binom{d}{j} H(m-1, f-j|d)}{H(m, f|d)}, \quad j = \max\{1, f - (m-1)d\}, \dots, \min\{d, f - m + 1\},$$

where $H(\cdot)$ follows the recursion: for any $0 < k \leq K$ and $0 \leq n \leq f$,

$$H(k, n|d) = \sum_{i=\max\{1, n-(k-1)d\}}^{\min\{d, n-k+1\}} \binom{d}{i} H(k-1, n-i|d), \quad H(1, n|d) = \binom{d}{n}.$$

B.1 PROOF OF LEMMA 3.1

Proof. Without loss of generality, suppose $\mathbf{u}$ and $\mathbf{u}'$ differ in the i -th dimension, and by the symmetry of DP, we can assume that $u_i = 1$ and $u'_i = 0$. We know that i is assigned to the $\lceil \text{mod}(\pi(i), d) \rceil$ -th bin. Among the K hash values, this change will affect all the bins that uses the data/hash of the $k^* = \lceil \text{mod}(\pi(i), d) \rceil$ -th bin (after permutation), both in the first scan (if it is non-empty) and in the densification process. Let N_{emp} be the number of empty bins in $h(\mathbf{u})$, and $\tilde{f}$ be the number of non-zero elements in the k^* -th bin. We have, for $x = 0, \dots, K - \lceil f/d \rceil$,

$$\begin{aligned} P(X = x) &= \sum_{j=\max(0, K-f)}^{K-\lceil f/d \rceil} \sum_{z=1}^{\min(f, d)} P(X = x | \tilde{f} = z, N_{emp} = j) P(\tilde{f} = z, N_{emp} = j) \\ &= \sum_{j=\max(0, K-f)}^{K-\lceil f/d \rceil} \sum_{z=1}^{\min(f, d)} P(X = x | \tilde{f} = z, N_{emp} = j) P(\tilde{f} = z | K - j) P(N_{emp} = j), \end{aligned}$$

where $P(\tilde{f} = z | K - j)$ is given in Lemma B.2 and $P(N_{emp} = j)$ can be calculated by Lemma B.1. To compute the first conditional probability, we need to compute the number of times the k^* -th bin is picked to generated hash values, and the hash values are different for $\mathbf{u}$ and $\mathbf{u}'$. Conditional on $\{\tilde{f} = z, N_{emp} = j\}$, denote $\Omega = \{k : \mathcal{B}_k \text{ is empty}\}$, and let R_k be the non-empty bin used for the k -th hash value $h_k(\mathbf{u})$, which takes value in $[K] \setminus \Omega$. We know that $|\Omega| = j$. We can write

$$X = \mathbb{1}\{h_{k^*}(\mathbf{u}) \neq h_{k^*}(\mathbf{u}')\} + \sum_{k \in \Omega} \mathbb{1}\{R_k = k^*, h_k(\mathbf{u}) \neq h_k(\mathbf{u}')\}.$$

Here we separate out the first term because the k^* -th hash always uses the k^* -bin. Note that the densification bin selection is uniform, and the bin selection is independent of the permutation for hashing. For the fixed densification, since the hash value $h_{k^*}(\mathbf{u})$ is generated and used for all hash values that use $\mathcal{B}_{k^*}$, we have

$$P(X = x | \tilde{f} = z, N_{emp} = j) = \mathbb{1}\{x = 0\} (1 - P_{\neq}) + \mathbb{1}\{x > 0\} P_{\neq} \cdot g_{bino}\left(x-1; \frac{1}{K-j}, j\right),$$

where $g_{bino}(x; p, n)$ is the probability mass function of the binomial distribution with n trials and success rate p , and $P_{\neq} = P(h_{k^*}(\mathbf{u}) \neq h_{k^*}(\mathbf{u}')) = (1 - \frac{1}{2^b}) \frac{1}{z}$. Based on the same reasoning, for re-randomized densification, we have

$$P(X = x | \tilde{f} = z, N_{emp} = j) = (1 - P_{\neq}) \cdot g_{bino}\left(x; \frac{P_{\neq}}{K-j}, j\right) + P_{\neq} \cdot g_{bino}\left(x-1; \frac{P_{\neq}}{K-j}, j\right).$$

Combining all the parts together completes the proof. $\square$

B.2 PROOF OF THEOREM 3.2

Proof. Let $\mathbf{u}$ and $\mathbf{u}'$ be neighbors only differing in one element. Denote $S = \{k \in [K] : h_k(\mathbf{u}) \neq h_k(\mathbf{u}')\}$ and $S^c = [K] \setminus S$. As discussed before, we can verify that for $k \in S^c$, we have $\frac{P(\tilde{h}_k(\mathbf{u})=i)}{P(\tilde{h}_k(\mathbf{u}')=i)} = 1$ for any $i = 0, \dots, 2^b - 1$. For $k \in S$, $e^{-\epsilon'} \leq \frac{P(\tilde{h}_k(\mathbf{u})=i)}{P(\tilde{h}_k(\mathbf{u}')=i)} \leq e^{\epsilon'}$ holds for any $i = 0, \dots, 2^b - 1$. Thus, for any $Z \in \{0, \dots, 2^b - 1\}^K$, the absolute privacy loss can be bounded by

$$\left| \log \frac{P(\tilde{h}(\mathbf{u}) = Z)}{P(\tilde{h}(\mathbf{u}') = Z)} \right| = \left| \log \prod_{k \in S} \frac{P(\tilde{h}_k(\mathbf{u}) = i)}{P(\tilde{h}_k(\mathbf{u}') = i)} \right| \leq |S|\epsilon' = |S|\frac{\epsilon}{N}. \quad (6)$$

By Lemma 3.1, with probability $1 - \delta$, $|S| \leq F_{fix}^{-1}(1 - \delta) = N$ for DP-OPH-fix; $|S| \leq F_{re}^{-1}(1 - \delta) = N$ for DP-OPH-re. Hence, (6) is bounded by ϵ with probability $1 - \delta$. This proves the (ϵ, δ) -DP. $\square$

B.3 PROOF OF THEOREM 3.3

Proof. The proof is similar to the proof of Theorem 3.2. Since the original hash vector $h(\mathbf{u})$ is not densified, there only exists exactly one hash value such that $h_k(\mathbf{u}) \neq h_k(\mathbf{u}')$ may happen for $\mathbf{u}'$ that differs in one element from $\mathbf{u}$. W.l.o.g., assume $u_i = 1$ and $u'_i = 0$, and $i \in \mathcal{B}_k$. If bin k is non-empty for both $\mathbf{u}$ and $\mathbf{u}'$ (after permutation), then for any $Z \in \{0, \dots, 2^b - 1\}^K$, $\left| \log \frac{P(\tilde{h}(\mathbf{u})=Z)}{P(\tilde{h}(\mathbf{u}')=Z)} \right| \leq \epsilon$ according to our analysis in Theorem 3.2 (the probability of hash in $[K] \setminus \{k\}$ cancels out). If bin k is empty for $\mathbf{u}'$, since $1 \leq \frac{e^\epsilon}{e^\epsilon + 2^b - 1} / \frac{1}{2^b} \leq e^\epsilon$ and $e^{-\epsilon} \leq \frac{1}{2^b} / \frac{1}{e^\epsilon + 2^b - 1} \leq 1$, we also have $\left| \log \frac{P(\tilde{h}(\mathbf{u})=Z)}{P(\tilde{h}(\mathbf{u}')=Z)} \right| \leq \epsilon$. Therefore, the algorithm is ϵ -DP as claimed. $\square$

B.4 PROOF OF THEOREM 3.5

Proof. For the two densified DP-OPH variants, DP-OPH-fix and DP-OPH-re, and the DP MinHash (DP-MH) methods, each full-precision (and unprivatized) hash value of $h(\mathbf{u})$ and $h(\mathbf{v})$ has collision probability equal to $P(h(\mathbf{u}) = h(\mathbf{v})) = J(\mathbf{u}, \mathbf{v})$. Let $h^{(b)}(\mathbf{u})$ denote the b -bit hash values. Since we assume the last b bits are uniformly assigned, we have $P(h^{(b)}(\mathbf{u}) = h^{(b)}(\mathbf{v})) = J + (1 - J)\frac{1}{2^b}$. Denote $p = \frac{\exp(\epsilon/N)}{\exp(\epsilon/N) + 2^b - 1}$. By simple probability calculation, the privatized b -bit hash values has collision probability

$$\begin{aligned} P(\tilde{h}(\mathbf{u}) = \tilde{h}(\mathbf{v})) &= P(\tilde{h}(\mathbf{u}) = \tilde{h}(\mathbf{v}) | h^{(b)}(\mathbf{u}) = h^{(b)}(\mathbf{v})) P(h^{(b)}(\mathbf{u}) = h^{(b)}(\mathbf{v})) \\ &\quad + P(\tilde{h}(\mathbf{u}) = \tilde{h}(\mathbf{v}) | h^{(b)}(\mathbf{u}) \neq h^{(b)}(\mathbf{v})) P(h^{(b)}(\mathbf{u}) \neq h^{(b)}(\mathbf{v})) \\ &= \left[p^2 + \frac{(1-p)^2}{2^b - 1} \right] \left(\frac{1}{2^b} + \frac{2^b - 1}{2^b} J \right) + \left[\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right] \left(\frac{2^b - 1}{2^b} - \frac{2^b - 1}{2^b} J \right) \\ &= \left[p^2 + \frac{(1-p)^2}{2^b - 1} - \frac{2p(1-p)}{2^b - 1} - \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right] \frac{2^b - 1}{2^b} J \\ &\quad + \frac{1}{2^b} \left[p^2 + \frac{(1-p)^2}{2^b - 1} + 2p(1-p) + \frac{2^b - 2}{2^b - 1} (1-p)^2 \right] \\ &= \left[p^2 + \frac{(1-p)^2 - 2(2^b - 1)p(1-p)}{(2^b - 1)^2} \right] \frac{2^b - 1}{2^b} J + \frac{1}{2^b} [p^2 + 2p(1-p) + (1-p)^2] \\ &= \frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b}, \end{aligned}$$

which implies $J = \frac{(2^b - 1)(2^b P(\tilde{h}(\mathbf{u}) = \tilde{h}(\mathbf{v})) - 1)}{(2^b p - 1)^2}$. Therefore, let $\hat{J} = \frac{1}{K} \sum_{k=1}^K \mathbb{1}\{\tilde{h}_k(\mathbf{u}) = \tilde{h}_k(\mathbf{v})\}$, then an unbiased estimator of J can be formulated as

$$\hat{J}_{unbias} = \frac{(2^b - 1)(2^b \hat{J} - 1)}{(2^b p - 1)^2}.$$

$\square$

B.5 PROOF OF THEOREM 3.6

Proof. As before, define $\hat{J} = \frac{1}{K} \sum_{k=1}^K \mathbb{1}\{\tilde{h}_k(u) = \tilde{h}_k(v)\}$. For all three methods, we know that $\mathbb{E}[\hat{J}] = \frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b}$. Denote $J_B = P(h^{(b)}(\mathbf{u}) = h^{(b)}(\mathbf{v})) = J + (1 - J)\frac{1}{2^b}$. $\hat{J}_{unbias} = \frac{(2^b - 1)(2^b \hat{J} - 1)}{(2^b p - 1)^2}$.

MinHash. We have

$$\begin{aligned} \text{Var}[\hat{J}] &= \mathbb{E}[\hat{J}^2] - \mathbb{E}[\hat{J}]^2 \\ &= \frac{1}{K^2} \mathbb{E} \left[\sum_{i=1}^K \mathbb{1}\{\tilde{h}_i(u) = \tilde{h}_i(v)\} + \sum_{i \neq j} \mathbb{1}\{\tilde{h}_i(u) = \tilde{h}_i(v)\} \mathbb{1}\{\tilde{h}_j(u) = \tilde{h}_j(v)\} \right] - \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right)^2 \\ &= \frac{1}{K} \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right) + \frac{K-1}{K} A - \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right)^2, \end{aligned}$$

where $A = \mathbb{E}[\mathbb{1}\{\tilde{h}_i(u) = \tilde{h}_i(v), \tilde{h}_j(u) = \tilde{h}_j(v)\}]$ for $i \neq j$. The key is to calculate A . By symmetry,

$$\begin{aligned} A &= P(\tilde{h}_i(u) = \tilde{h}_i(v), \tilde{h}_j(u) = \tilde{h}_j(v) | h_i^{(b)}(u) = h_i^{(b)}(v), h_j^{(b)}(u) = h_j^{(b)}(v)) P(h_i^{(b)}(u) = h_i^{(b)}(v), h_j^{(b)}(u) = h_j^{(b)}(v)) \\ &\quad + 2P(\tilde{h}_i(u) = \tilde{h}_i(v), \tilde{h}_j(u) = \tilde{h}_j(v) | h_i^{(b)}(u) = h_i^{(b)}(v), h_j^{(b)}(u) \neq h_j^{(b)}(v)) P(h_i^{(b)}(u) = h_i^{(b)}(v), h_j^{(b)}(u) \neq h_j^{(b)}(v)) \\ &\quad + P(\tilde{h}_i(u) = \tilde{h}_i(v), \tilde{h}_j(u) = \tilde{h}_j(v) | h_i^{(b)}(u) \neq h_i^{(b)}(v), h_j^{(b)}(u) \neq h_j^{(b)}(v)) P(h_i^{(b)}(u) \neq h_i^{(b)}(v), h_j^{(b)}(u) \neq h_j^{(b)}(v)) \\ &:= A_{11} + 2A_{01} + A_{00}. \end{aligned}$$

By independence, we have

$$\begin{aligned} A_{11} &= \left(p^2 + \frac{(1-p)^2}{2^b - 1} \right)^2 J_B^2 \\ A_{10} &= \left(p^2 + \frac{(1-p)^2}{2^b - 1} \right) \left(\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right) J_B(1 - J_B) \\ A_{00} &= \left(\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right)^2 (1 - J_B)^2, \end{aligned}$$

which leads to

$$A = \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right)^2.$$

Thus, we have

$$\text{Var}[\hat{J}] = \frac{1}{K} \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right) \left(\frac{2^b - 1}{2^b} - \frac{(2^b p + 1)^2}{2^b(2^b - 1)} J \right)$$

and

$$\begin{aligned} \text{Var}[\hat{J}_{unbias, MH}] &= \frac{2^{2b}(2^b - 1)^2}{(2^b p - 1)^4} \text{Var}[\hat{J}] \\ &= \frac{1}{K} \left(\frac{(2^b p + 1)^2}{(2^b p - 1)^2} J + \frac{2^b - 1}{(2^b p - 1)^2} \right) \left(\frac{(2^b - 1)^2}{(2^b p - 1)^2} - \frac{(2^b p + 1)^2}{(2^b p - 1)^2} J \right). \end{aligned}$$

DP-OPH-fix. We write $\hat{J} = \frac{1}{K} \sum_{k=1}^K (\tilde{I}_k^N + \tilde{I}_k^E)$, where $\tilde{I}_k^N$ is the indicator function of hash collision at the k -th bin and when the bin is non-empty, and $\tilde{I}_k^E$ is the indicator function of hash

collision at the k -th bin and when the bin is empty. Similar to previous analysis,

$$\begin{aligned} \text{Var}[\hat{J}] &= \frac{1}{K^2} \mathbb{E} \left[\left(\sum_{k=1}^K (\tilde{I}_k^N + \tilde{I}_k^E) \right)^2 \right] - \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right)^2 \\ &= \frac{1}{K} \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right) + \frac{1}{K^2} A - \left(\frac{(2^b p + 1)^2}{2^b(2^b - 1)} J + \frac{1}{2^b} \right)^2, \end{aligned}$$

where

$$\begin{aligned} A &= \mathbb{E} \left[\sum_{i \neq j} (\tilde{I}_i^N + \tilde{I}_i^E)(\tilde{I}_j^N + \tilde{I}_j^E) \right] \\ &= \mathbb{E}_m \left[\mathbb{E}[m(m-1)\tilde{I}_i^N \tilde{I}_j^N + 2m(K-m)\tilde{I}_i^N \tilde{I}_j^E + (K-m)(K-m-1)\tilde{I}_i^E \tilde{I}_j^E | m] \right]. \quad (7) \end{aligned}$$

Here the condition on “ $\cdot | m$ ” means the event that there are m simultaneously non-empty bins. Denote $I_k = \mathbb{1}\{h_k(u) = h_k(v)\}$ be the collision indicator of the original hash values, and $I_k^{(b)} = \mathbb{1}\{h_k^{(b)}(u) = h_k^{(b)}(v)\}$ be the collision indicator of the b -bit hash values. For two non-empty bins i and j , we have

$$\begin{aligned} \tau_{11} &:= P(h_i(u) = h_i(v), h_j(u) = h_j(v) | m) = \mathbb{E}[I_i I_j | m] = J\tilde{J}, \\ \tau_{10} &:= P(h_i(u) = h_i(v), h_j(u) \neq h_j(v) | m) = \mathbb{E}[I_i(1 - I_j) | m] = J - J\tilde{J}, \\ \tau_{00} &:= P(h_i(u) \neq h_i(v), h_j(u) \neq h_j(v) | m) = \mathbb{E}[(1 - I_i)(1 - I_j) | m] = 1 - 2J + J\tilde{J}, \end{aligned}$$

and using total probability formula (conditional on h_i and h_j),

$$\begin{aligned} P(h_i^{(b)}(u) = h_i^{(b)}(v), h_j^{(b)}(u) = h_j^{(b)}(v) | m) &= \mathbb{E}[I_i^{(b)} I_j^{(b)} | m] \\ &= \tau_{11} + 2\frac{1}{2^b} \tau_{10} + \frac{1}{2^{2b}} \tau_{00} \\ &= J\tilde{J} + \frac{1}{2^{b-1}}(J - J\tilde{J}) + \frac{1}{2^{2b}}(1 - 2J + J\tilde{J}) := P_{11} \end{aligned}$$

$$P(h_i^{(b)}(u) = h_i^{(b)}(v), h_j^{(b)}(u) \neq h_j^{(b)}(v) | m) = \mathbb{E}[I_i^{(b)}(1 - I_j^{(b)}) | m] = J_B - P_{11} := P_{10}$$

$$P(h_i^{(b)}(u) \neq h_i^{(b)}(v), h_j^{(b)}(u) \neq h_j^{(b)}(v) | m) = \mathbb{E}[(1 - I_i^{(b)})(1 - I_j^{(b)}) | m] = 1 - 2J_B + P_{11} := P_{00}.$$

Thus,

$$\begin{aligned} \mathbb{E}[\tilde{I}_i^N \tilde{I}_j^N | m] &= \left(p^2 + \frac{(1-p)^2}{2^b - 1} \right)^2 P_{11} + 2 \left(p^2 + \frac{(1-p)^2}{2^b - 1} \right) \left(\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right) P_{10} \\ &\quad + \left(\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right)^2 P_{00}. \end{aligned}$$

For two empty bins i and j , we have, for fixed densification,

$$\tau_{11,f} = P(h_i(u) = h_i(v), h_j(u) = h_j(v) | m) = \frac{1}{m} J + \frac{m-1}{m} J\tilde{J},$$

$$\tau_{10,f} = P(h_i(u) = h_i(v), h_j(u) \neq h_j(v) | m) = \mathbb{E}[I_i(1 - I_j)] = \frac{m-1}{m} (J - J\tilde{J}),$$

$$\tau_{00,f} = P(h_i(u) \neq h_i(v), h_j(u) \neq h_j(v) | m) = \mathbb{E}[(1 - I_i)(1 - I_j)] = 1 - (2 - \frac{1}{m})J + \frac{m-1}{m} J\tilde{J}.$$

Similarly,

$$\begin{aligned} \mathbb{E}[\tilde{I}_i^E \tilde{I}_j^E | m] &= \left(p^2 + \frac{(1-p)^2}{2^b - 1} \right)^2 P_{11,f} + 2 \left(p^2 + \frac{(1-p)^2}{2^b - 1} \right) \left(\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right) P_{10,f} \\ &\quad + \left(\frac{2p(1-p)}{2^b - 1} + \frac{2^b - 2}{(2^b - 1)^2} (1-p)^2 \right)^2 P_{00,f}, \end{aligned}$$

where

$$P_{11,f} = \tau_{11,f} + \frac{1}{2^{b-1}} \tau_{10,f} + \frac{1}{2^{2b}} \tau_{00,f}, \quad P_{10,f} = J_B - P_{11,f}, \quad P_{00,f} = 1 - 2J_B + P_{11,f}.$$

It is not hard to note that $\mathbb{E}[\tilde{I}_i^N \tilde{I}_j^E | m] = \mathbb{E}[\tilde{I}_i^E \tilde{I}_j^N | m]$. Putting pieces together into (7), we get the variance for DP-OPH-fix.

DP-OPH-re. For DP-OPH-re, most calculations are the same as DP-OPH-fix. According to Li et al. (2019), we have

$$\begin{aligned}\tau_{11,r} &= P(h_i(u) = h_i(v), h_j(u) = h_j(v)|m) = \mathbb{E}[I_i I_j] = \frac{\zeta(m)}{m} J + \frac{m - \zeta(m)}{m} J \tilde{J}, \\ \tau_{10,r} &= P(h_i(u) = h_i(v), h_j(u) \neq h_j(v)|m) = \mathbb{E}[I_i (1 - I_j)] = \frac{m - \zeta(m)}{m} (J - J \tilde{J}), \\ \tau_{00,r} &= P(h_i(u) \neq h_i(v), h_j(u) \neq h_j(v)|m) = \mathbb{E}[(1 - I_i)(1 - I_j)] \\ &= 1 - (2 - \frac{\zeta(m)}{m}) J + \frac{m - \zeta(m)}{m} J \tilde{J},\end{aligned}$$

with $\zeta(m) = \mathbb{E}[\frac{1}{f}|m]$ where the conditional distribution of $\tilde{f}$ is given in Lemma B.2. We then get $P_{11,r}$, $P_{10,r}$, $P_{00,r}$ correspondingly. Plugging them into the formula above completes the proof. $\square$